

A Cheatsheet for Python Packaging with uv

Jianhua Liu

Fall 2026

Contents

1	Introduction	3
1.1	The <code>pyproject.toml</code> file	3
1.2	Lockfile and synchronization	4
2	Installation of uv	4
2.1	Installation of uv on Windows	4
2.1.1	Checking where uv is installed on Windows	4
2.2	Installation of uv on Unix-like systems	4
2.3	Verifying and updating the uv installation	4
3	Basic uv operations for various tasks	4
3.1	Checking the uv commands and help	5
3.2	Working with a Python interpreter	5
3.2.1	Viewing installed Python versions	5
3.2.2	Installing a new Python interpreter	5
3.2.3	Reinstalling Python	6
3.2.4	Uninstalling a Python interpreter	6
3.2.5	Pinning a Python version	6
3.2.6	Checking the Python version of a project	6
3.2.7	Changing Python version in a project	6
3.2.8	Selecting a Python interpreter in VS Code	6
3.3	Running a Python script	6
3.3.1	Running a Python script without dependencies	6
3.4	Managing dependencies for a project	7
3.4.1	Checking the packages installed	7
3.4.2	Removing a dependency	7
3.4.3	Updating dependency to a newer version	7
3.4.4	Installing dependencies in two approaches	7
3.4.5	Installing a local development package	7
3.5	Working with lockfile for synchronization	7
3.5.1	Creating and updating <code>uv.lock</code>	8

3.5.2	Creating and updating <code>pylock.toml</code> or <code>requirements.txt</code>	8
4	Working with tools/apps	8
4.1	Managing an external tool with uv	8
4.1.1	Installing an external tool with pipx	8
4.1.2	Running an external tool with uv without installation	8
4.1.3	Installing an external tool with uv	9
4.1.4	Running an external tool installed with uv	9
4.1.5	Upgrading external tools installed with uv	9
4.2	Managing local apps	9
4.2.1	Installing a local app	9
4.2.2	Running a local app	9
4.2.3	Uninstalling a tool with uv	9
5	Workflow for scratch-created projects	9
5.1	Creating a project	9
5.1.1	Creating a project for multiple scripts	10
5.1.2	Creating a project for a CLI app	10
5.1.3	Creating a library project	10
5.1.4	Creating a workspace	10
5.2	Adding dependencies to a scratch-created project	10
5.2.1	Adding conventional PyPI packages	11
5.2.2	Adding dependencies from GitHub or other online recourses	11
5.2.3	Adding a local file	11
5.3	Adding dependencies to special groups	11
5.4	Addressing issues with running a script in the project	11
5.4.1	Manually activating the virtual environment for a project with uv	11
5.4.2	Activating the virtual environment for a project with code	12
5.5	Additional tricks and solutions	12
5.5.1	Renaming the executable	12
5.5.2	Running pytest	12
6	Workflow for cloned projects	12
6.1	Working with cloned uv-managed projects	13
6.2	Managing cloned projects created by other tools to work under uv	13
6.3	Managing cloned projects with <code>requirements.txt</code> to work under uv	13
6.3.1	Creating a simple <code>pyproject.toml</code> file	13
6.3.2	Adding dependencies manually	14
6.3.3	Adding dependencies from <code>requirements.txt</code>	14
6.4	Working on cloned projects with <code>requirements.txt</code> as it	14
7	Packaging an exemplary project with external dependency	14
7.1	Creating the project	14
7.2	Editing source files	14
7.2.1	Editing the <code>README.md</code> file	15
7.2.2	Editing or creating source files in the <code>src/numreps</code> folder	16
7.2.3	Creating source files in the <code>tests</code> folder	19

7.3	Adding dependencies	21
7.4	Running the tests	21
7.5	Running the project	21
8	Packaging an exemplary project with local dependency	21
8.1	Creating the project	21
8.2	Editing source files	21
8.3	Adding dependencies	24
8.4	Installing the project	24
8.5	Running the project	24

1 Introduction

uv is a fast Python package and project management tool developed by Astral. It can manage dependencies, virtual environments, Python versions, and project packaging through a unified workflow (to build and publish our projects as well as install and run Python projects from various as command line tool). Designed as a modern alternative to combining tools such as pip, venv, and virtualenv, uv emphasizes speed, reproducibility, and simplicity.

If you are curious where comes the name of uv, you can consider this rumor: it's name comes from ultraviolet, which has a shorter wavelength or higher frequency than the visible light, implying that uv is faster than all the other existing python packaging tools.

There are a few types of files that are important with uv. They are introduced below.

1.1 The `pyproject.toml` file

uv uses the `pyproject.toml` file to manage the dependencies. It defines build system requirements and configuration for tools. It also defines metadata and others. It aims to provide a unified place for project metadata and dependency management. Below is an example.

```

1  [build-system]
2  requires = ["setuptools", "wheel"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "your_project"
7  version = "0.1.0"
8  description = "A sample Python project"
9  authors = [
10     { name = "Your Name", email = "your.email@example.com" }
11  ]
12  dependencies = [
13     "requests>=2.24.0",
14     "numpy>=1.19.2",
15  ]
16
17  [project.optional-dependencies]
18  dev = [
19     "pytest",
20  ]

```

1.2 Lockfile and synchronization

Lockfiles can be used for synchronizing the installation of dependencies across different computers for a specific project.

Different types of lockfiles are used to specify the exact versions of each package as will be discussed later.

2 Installation of uv

Installation steps are detailed at <https://docs.astral.sh/uv/getting-started/installation/>.

After the installation, we can run `uv --version` to verify it runs.

2.1 Installation of uv on Windows

Use `irm` to download the script and execute it with `iex`. This can be done by executing the following in a Windows terminal:

```
irm https://astral.sh/uv/install.ps1 | iex
```

Or, you can download `uv-x86_64-pc-windows-msvc.zip` from <https://github.com/astral-sh/uv/releases> to do a manual installation.

2.1.1 Checking where uv is installed on Windows

To see where uv is installed, we can install a special Python version and see its location. Assume we have run `uv python install 3.12` before. Then, we can see installation using `uv python list`. It shows that version 3.12.12 is installed at `C:\Users\jhl\AppData\Roaming\uv\python\cpython-3.12.12-windows-x86_64-none\python.exe`.

2.2 Installation of uv on Unix-like systems

On Unix-like systems, including Linux, macOS, and WSL, to install uv, open up a terminal and run

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

It is installed in your home directory; here is an example: `/home/jhl/.local/bin`.

2.3 Verifying and updating the uv installation

To verify the installation, we can check the version of uv by running `uv self version` or `uv --version`.

New versions of uv are released very frequently. To update uv, run `uv self update` in a terminal.

3 Basic uv operations for various tasks

uv is a feature-rich tool which can be used to replace a large number other tools. Yet, we only discuss a subset of features here, focusing on using basic uv operations for different tasks not related to project development. Operations related to project development are discussed in later sections.

If you want to see a list of detailed features of uv, see <https://docs.astral.sh/uv/getting-started/features/>, which are listed in the following categories:

- Python version management.
- Using standalone Python scripts.
- Managing Python-based projects. This is the focus of this doc, where we develop projects for our own or others to use.
- Running and installing tools. This is about how to use projects developed by us or others on our system for various purposes.
- The pip interface. pip is a very popular Python dependence handling tool. uv can be used as a drop-in replacement of pip.
- Utility. This is for the management of uv itself.

All the commands below will run in a terminal, either on the Unix-like systems or on Windows.

3.1 Checking the uv commands and help

As an all-in-one tool, uv has a large collection of commands. It is neither possible nor necessary to remember all commands and their options. What we can do is to seek help when needed.

- To check the list of uv commands, run `uv`.
- To get a brief help of all uv commands, run `uv --help`.
- To get a more detailed help (in theory) of all uv commands, run `uv help`. Note that this is on the same level as `run --help`.
- To get detailed help on each command, run `uv help <command>`. To navigate, use PgUp or PgDn. To exit, press “q”.

3.2 Working with a Python interpreter

3.2.1 Viewing installed Python versions

To see all the versions that can be used by uv, including locally installed and remotely available, run

```
1 uv python list
```

3.2.2 Installing a new Python interpreter

There are *two* approaches to install a new Python interpreter. The *explicit* approach is running

```
1 uv python install 3.13
```

- If we ignore the version, the newest version will be installed.
- We can install multiple versions in one-shot.

The *implicit* approach is to create a project with a specified version, as will be shown below.

For the newer versions of Python, where two version, regular and freethreaded versions available, if we want to use the later, we can use `uv python install 3.14+freethreaded`.

3.2.3 Reinstalling Python

For uv-managed Python, we can upgrade to a new release, say from 3.12.9 to 3.12.12. Here we use

- `uv python install 3.12--reinstall.`

3.2.4 Uninstalling a Python interpreter

We can remove a Python 3.12 installation using `uv python uninstall 3.12.`

3.2.5 Pinning a Python version

To use a specific Python version, say 3.11, run

```
uv python pin 3.11
```

Note that on Windows, we may have access issues.

3.2.6 Checking the Python version of a project

There are two ways to see the version of Python used with **a specific project**:

- Checking the `.python-version` file.
- Printing out info in a script `print(sys.version).`

3.2.7 Changing Python version in a project

This can be done by editing the python version in the `.python-version` file directly.

Note that after this change, when we run `uv run ...`, the old v-env of the project will be removed and a new one will be created.

(Note that we may have to change the Python version in the `requires-python` field of `pyproject.toml`. Need to verify this later.)

3.2.8 Selecting a Python interpreter in VS Code

When we use VS Code, we need to select the Python interpreter from uv. This can be done with a setup in VS Code.

- Open the VS Code Command Palette (Ctrl/Cmd + Shift + P).
- Type “Python: Select Interpreter”.
- Choose the interpreter from the uv virtual environment, which should be something like `.venv/bin/python` or similar.

3.3 Running a Python script

Note that a uv-installed Python interpreter may not be usable directly in CLI. That is Ok as we will usually run Python scripts by using `uv run`, as discussed here.

3.3.1 Running a Python script without dependencies

uv can be used to run a standalone Python script, say `hello_world.py`, directly with `uv run hello_world.py`. Yet, we don’t see any advantages as compared to just running `python hello_world.py`.

3.4 Managing dependencies for a project

We will discuss, in later sections, workflows for two different types of projects: those created locally from scratch and those cloned from somewhere else. Here we discuss the common operations we perform to address dependencies on an average project.

3.4.1 Checking the packages installed

To list the packages installed, we can use

```
1 uv pip list # for a plain list or
2 uv tree    # a tree-shape list
```

3.4.2 Removing a dependency

This can be done using

```
1 uv remove <package>
```

3.4.3 Updating dependency to a newer version

This can be done by running `uv lock --upgrade`.

Note that:

- We may have to edit the `pyproject.toml` file manually to remove some *less-than constraints*, which limits us to use very new versions, before running the command.
- The above instruction will upgrade the `uv.lock` file only. It does not update `pyproject.toml` file.

To upgrade the installed dependencies, we need to run `uv sync`.

3.4.4 Installing dependencies in two approaches

There are two approaches to add dependencies:

- `uv add <a_package>`. This will install the dependency and update the `pyproject.toml` file; this is the preferred approach.
- `uv pip install <a_package>`. This will install the dependency only without updating the `pyproject.toml` file.

3.4.5 Installing a local development package

If we have a local project, developing a package to be published later, we need to be able to test the component of the package during the development. Here, we need to run `uv pip install -e .` so that the package can be used as if it is an dependency.

Note that this is not the same as the installation of a local app, which is discussed at Installing a local app.

3.5 Working with lockfile for synchronization

We have the following types of lockfiles:

- `uv.lock` is a *universal* or *cross-platform* lockfile that captures the packages that would be installed across all possible Python markers such as operating system, architecture, and Python version. Note that this is used by uv only.
- `pylock.toml` is a new Python standard.
- `requirements.txt` is the widely used one currently.

3.5.1 Creating and updating `uv.lock`

`uv.lock` will be created or updated from `pyproject.toml` by the following command:

```
1 uv lock
```

3.5.2 Creating and updating `pylock.toml` or `requirements.txt`

This can be done by the `uv export` command from `pyproject.toml`. Note that the project is re-locked before exporting unless the `--locked` or `--frozen` flag is used.

```
1 uv export -o pylock.toml
2 uv export -o requirements.txt
```

Note that if we have `requirements.in`, we can create `pylock.toml` or `requirements.txt` by running

```
1 uv pip compile -o pylock.toml -r requirements.in
```

4 Working with tools/apps

A tool is a stand-alone Python app that we can run in CLI. We can have two types of tools:

- An external one that is not developed by us, which can come from various sources, such as PyPI.
- A local one that we have the source files stored locally. This can be developed by us or cloned from GitHub.

Here, we use `ruff` as an example.

4.1 Managing an external tool with uv

4.1.1 Installing an external tool with `pipx`

For comparison, let's see what is the approach we use before the uv era.

`pipx` was created to install a tool in an isolated v-env. For example, we can install `ruff` using `pipx install ruff`. After that, we can run `ruff` directly anywhere.

4.1.2 Running an external tool with uv without installation

uv runs very fast, and it is ok to run an external tool without installing it. For example, we can run `ruff` via `uvx ruff`, which is a short cut of `uv tool run ruff`.

If you want to see more details about running a tool without installing it, read <https://docs.astral.sh/uv/guides/tools/#using-tools>.

4.1.3 Installing an external tool with uv

For most of the tools, we want to install them locally before running them. To install, run `uv tool install ruff`. Note that this command just replaces `pipx` with `uv tool`.

4.1.4 Running an external tool installed with uv

After installation using `uv tool install <command>`, we can run the tool by typing `command` anywhere in CLI.

4.1.5 Upgrading external tools installed with uv

To upgrade an external tool installed with uv, run `uv tool upgrade ruff`.

To upgrade all tools, run `uv tool upgrade --all`.

4.2 Managing local apps

Suppose we have a local CLI app, saved in `<app_proj>`, which is developed locally or cloned from GitHub.

4.2.1 Installing a local app

Like with an external tool, we can install it using uv and run it anywhere in our system without having to activate the v-env.

To install, go to the root folder of the project and run `uv tool install . -e`.

Sometimes, especially after installing new dependencies, we need to run it again **to force some changes**. This can be done by running `uv tool install . -e --force`.

4.2.2 Running a local app

Once the local app is installed, it runs in the same way as all other CLI apps.

4.2.3 Uninstalling a tool with uv

We can uninstall an installed tool with uv as well. For both external tools and local apps, we use the same instruction:

```
uv tool uninstall <a_tool>
```

5 Workflow for scratch-created projects

Here, we discuss the uv workflow when working on a project we create from scratch locally, regardless it is hosted on GitHub or not.

5.1 Creating a project

Different Python projects can be used for different purposes in addition to their different functionality. For example, we can create a project that is targeted for:

- A **collection of scripts**, including various CLI and GUI apps, which can be hold in various subfolders of the root project folder.
- A **CLI app** that can run anywhere on our computer. Note that this type of project can also include **GUI apps** that can run anywhere on our computer.
- A Python package that is used as a library.
- A **workspace** to host related Python packages, including extensions (in other languages, such as Rust). Note that workspace is a concept used commonly in Rust but not in Python. As uv is developed in Rust, uv developers borrowed this concept to uv to handle workspace as well.

5.1.1 Creating a project for multiple scripts

Many times, we will have multiple scripts related to a certain topic, such as all projects of a class. For organization reasons, we prefer to have all these scripts in a single project folder with dependencies handled together and multiple scripts in different subfolders.

The creation of such a project, called `<script_proj>`, can be done by running `uv init <script_proj>`. After the running, we will see that there are five files created in `<script_proj>` (say, `proj_scripts`): `.gitignore`, `.python-version`, `README.md`, `main.py`, and `pyproject.toml`. There is also a `.git` folder created.

Except the most trivial projects, we will have many scripts in such projects. To reduce the clutter, we will use subfolders under the root folder to host related scripts.

5.1.2 Creating a project for a CLI app

We can create a **packaged project** by running: `uv init --package <app_proj>`. Note that we can use `--python 3.12` to specify the version of Python to use.

When compared with the script project, an additional `src` folder is created to host the source files.

Note that uv cannot create a `tests` folder like Poetry does. To host the test scripts, we need to manually create the `tests` folder in the same level as the `src` folder.

5.1.3 Creating a library project

There is another option for the creation of a project, which is `--lib`, used to for hosting a library project: `uv init --lib <a_lib>`.

5.1.4 Creating a workspace

Currently, we don't have a use case yet. If we do, we will expend the guide according to <https://docs.astral.sh/uv/concepts/projects/workspaces/>.

5.2 Adding dependencies to a scratch-created project

Assume we have a project created from scratch by using uv and are in the phase of developing code. In this process, we may need to add additional dependencies.

To **add** such a dependency, we use `uv add <package>`. This will install the package in the `.venv` folder and update the `pyproject.toml` file.

We may have to use the *dependency source* to tell where to find a certain dependency, as summarized below. For more details, see <https://docs.astral.sh/uv/concepts/projects/dependencies/#dependency-sources>.

5.2.1 Adding conventional PyPI packages

For conventional PyPI packages, we normally don't need to do anything special.

We can specify the version of the package though. For example

```
1 uv add 'requests==2.31.0'
```

5.2.2 Adding dependencies from GitHub or other online recourses

If the package is host on GitHub, we can use

```
1 uv add git+https://github.com/psf/requests
```

5.2.3 Adding a local file

When we add a local dependency, such as files from a project we developed locally, we can install it by specifying the path of the project. Say, assume we have a package named `numreps` under `projects`. If we develop a new project named `numreps-repl` under `projects`, we need to use the following with the `numreps-repl` project to add `numreps` as dependency:

```
1 uv add ../numreps
```

After that, `numreps` will be used in the same way as the other dependencies.

5.3 Adding dependencies to special groups

uv supports adding dependencies to special groups. For example, if we want to add `pytest` in to the `dev` group, we will run `uv add --dev pytest`. In the `pyproject.toml` file, we will see the following:

```
1 [dependency-groups]
2 dev = [
3     "pytest>=8.3.4",
4 ]
```

For special groups, we can use `--dev`, `--group`, or `--optional` flags.

5.4 Addressing issues with running a script in the project

When we run `uv run a_script.py` under the root directory of the project, uv will automatically activate the v-env stored in `.venv`. If there is any issue with the importing of some modules, there are two approaches to handle the problem.

5.4.1 Manually activating the virtual environment for a project with uv

- On Unix-like systems, this can be done by running `source .venv/bin/activate`.
- On Windows, this can be done by running `.venv\Scripts\activate`.

For both OSES, to deactivate the v-env, run `deactivate`.

Note that this can be awkward if the folder we are running from is not the project folder. For this, we can use the following approach.

5.4.2 Activating the virtual environment for a project with code

This can be done in the `__init__.py` file, as shown below.

```
1 from pathlib import Path
2 import subprocess
3
4 from .app_cli import app
5
6 def activate_env():
7     script_dir = Path(__file__).parent
8     activate_path = script_dir / "../../.venv/bin/activate"
9     subprocess.run(f"source {activate_path}",
10                   shell=True, executable="/bin/bash")
11
12 def main():
13     activate_env()
14     app()
15
16 if __name__ == "__main__":
17     main()
```

5.5 Additional tricks and solutions

5.5.1 Renaming the executable

By default, the name of executable app is defined in `pyproject.toml`, as shown below:

```
1 [project.scripts]
2 mp_app = "mp_app:main"
```

We need to pay attention to the following:

- The first `mp_app` can be changed to another name if we want to **rename** the executable.
- The second `mp_app` cannot be changed as it defines the project (package).
- The `main` function is the entry point of the program. It must be defined in `__init__.py`.

5.5.2 Running pytest

To run pytest:

- Create a `tests` folder beside the `src` folder and put our test functions in files of that folder.
- Go to the root folder of the project.
- Activate the v-env of the project as described above.
- Run `pytest` to run all test functions or run `pytest tests/a_test_file.py` to run a specific file.

6 Workflow for cloned projects

A cloned project is a project we copied from other sources:

- It can come from `git clone`.
- It can also come from downloading via archive tools.

6.1 Working with cloned uv-managed projects

For these projects, they come with `pyproject.toml` and `uv.lock`. Here, we just need to run `uv run folder/script.py`, and the v-env will be created automatically and the dependencies will be installed. After that, the command will run.

Or, we can run `uv sync` to pre-install the dependencies used by the app.

6.2 Managing cloned projects created by other tools to work under uv

Some projects come with a `pyproject.toml` file; for example, Poetry managed projects have such a file. Unfortunately, this is not in the format that uv can use. We need to migrate a project managed by other tools to uv if we want to keep working on the project under uv.

There is a tool called `migrate-to-uv` (<https://github.com/mkniewallner/migrate-to-uv>), which can be used to migrate a project from the management of following tools to uv:

- Poetry
- Pipenv
- pip-tools
- pip

To use this tool:

- Change the directory to the project folder.
- Run `uvx migrate-to-uv` to create `pyproject.toml` and `uv.lock` files.
- Run `uv sync` to install the v-env.

6.3 Managing cloned projects with `requirements.txt` to work under uv

If we cannot use the above `migrate-to-uv` to migrate the project, we can do a manual conversion so that the project can be under management of uv.

6.3.1 Creating a simple `pyproject.toml` file

First, we need to create a simple `pyproject.toml` file, as shown below.

```
1 [project]
2 name = "your-project"
3 version = "0.1.0"
4 dependencies = [
5
6 ]
```

With this file, we don't need to create the `.venv` folder by running `uv venv`. This folder will be created automatically when we run the `uv add <package>` command.

6.3.2 Adding dependencies manually

For many small projects, we can use a tool, such as `pydeps`, to extract the dependencies and add them manually using `uv add <package>` for each dependency.

If the list of dependencies is not too long and the project does come with a `requirements.txt`, we can move on to the next step.

6.3.3 Adding dependencies from `requirements.txt`

This is done by running:

```
1 uv add -r requirements.txt
```

Note that the list of dependencies in the `pyproject.toml` is usually much longer than that created from the simple `uv add <a_package>` command.

6.4 Working on cloned projects with `requirements.txt` as it

If we want to work on the project without converting to uv managed, we can just install the dependencies with `uv pip install -r requirements.txt`.

Note that `uv add -r requirements.txt` does not work since we don't have the needed `pyproject.toml` file.

Yet, **before the above** installation, we need to create a v-env and activate it. Otherwise, the installation will be in the default Python env. So, the standard way for the installation has three steps:

```
1 uv venv
2 . .venv/bin/activate
3 uv pip install -r requirements.txt
```

7 Packaging an exemplary project with external dependency

This is an exemplary CLI app project for illustrating packaging with uv. Please follow the steps below to create the project. Here, we assume uv is installed.

This project is illustrated on Windows with notes on the Unix-like systems.

7.1 Creating the project

Go to the parent folder in a terminal where you want to place this project. Say, `C:\projects` (or `~/projects` on Unix-like systems).

Create the project by running:

```
1 uv init --package --python 3.12 numreps
```

7.2 Editing source files

Now, change to the project folder `numreps` in a terminal and start VS Code by running `code ..` You should see the following files created:

- `.gitignore`: the Git ignore files, used for ignoring source files when we use Git. This will not discuss the details of this file here for simplicity. A customized file is shown below.
- `.python-version`: the file showing the version of Python.
- `pyproject.toml`: the project/package management file.
- `README.md`: the standard readme file.

`.gitignore`:

```
1 # Ignore everything
2 *
3
4 # Unignore top-level essentials
5 !/.gitignore
6 !/.python-version
7 !/pyproject.toml
8 !/pytest.ini
9 !/README.md
10 !/uv.lock
11
12 # Unignore src/numreps
13 !/src/
14 !/src/numreps/
15 !/src/numreps/*.py
16
17 # Unignore tests
18 !/tests/
19 !/tests/*.py
```

7.2.1 Editing the `README.md` file

Copy the following contents between the text separators, which provides a detailed description to the exemplary project, to this readme file:

numreps stands for number representations, and this app is used to convert a number to different representations:

For the CLI application, we have the following cases:

- `numreps d2b -- n_bit num`: convert a number `num`, say 19, to `n_bit`-bit, say 8-bit, binary expression in the form of `0b0001_0011`. Here `num` is signed. If `num` is positive, we can ignore `--` in the command.
- `numreps d2h -- n_bit num`: convert a number `num`, say 19, to `n_bit`-bit, say 8-bit, hexadecimal expression in the form of `0x13`. Here `num` is signed. If `num` is positive, we can ignore `--` in the command.
- `numreps b2d type n_bit num`: convert a `n_bit`-bit binary number `num` to decimal. If `type = "u"`, then, the binary is seen as unsigned; if `type = "i"`, then the binary is seen as signed. Other values of `type` are not supported.

- `numreprs h2d type n_bit num`: convert a `n_bit`-bit hexadecimal number `num` to decimal. If `type` = "u", then, the corresponding binary is seen as unsigned; if `type` = "i", then the corresponding binary is seen as signed. Other values of `type` are not supported.

To see detailed format of how to run the app and updated functionalities of this app, run `numreprs --help`.

7.2.2 Editing or creating source files in the `src/numreprs` folder

We will use the following source files in this folder.

7.2.2.1 The `__init__.py` file

This is the entry point of the app.

`__init__.py`:

```
1 from numreprs.app_cli import app
2
3 def main():
4     app()
```

7.2.2.2 The `app_cli.py` file

This is the collection of commands for this app.

`app_cli.py`:

```
1 import typer
2 from numreprs.recipes import d2b_, d2h_, b2d_, h2d_
3
4
5 app = typer.Typer()
6
7 __version__ = "0.1.0"
8
9
10 #####
11 # Print out the version: numreprs --version
12 def print_version(value: bool):
13     if value:
14         typer.echo(f"Version of numreprs: {__version__}")
15         raise typer.Exit()
16
17
18 # Add the version option.
19 @app.callback()
20 def callback(version: bool = typer.Option(
21     None, "--version", callback=print_version, is_eager=True)):
22     pass
23
24
25 #####
26 # Decimal to n-bit binary
```

```

27 @app.command()
28 def d2b(n_bit: int, num: int):
29     """Print num, a decimal, in n-bit binary format: numreps d2b -- n_bit num.
30     Example: numreps d2b -- 32 -66. If num is positive, -- can be removed."""
31     binary = d2b_(n_bit, num)
32     typer.echo(f'{num} to {n_bit}-bit binary: {binary}')
33
34
35 # Decimal to n-bit hexadecimal
36 @app.command()
37 def d2h(n_bit: int, num: int):
38     """Print num, a decimal, in n-bit binary format: numreps d2h -- n_bit num.
39     Example: numreps d2h -- 32 -66. If num is positive, -- can be removed."""
40     hexa_d = d2h_(n_bit, num)
41     typer.echo(f'{num} to {n_bit}-bit binary: {hexa_d}')
42
43
44 # n-bit binary to decimal
45 @app.command()
46 def b2d(b_type: str, n_bit: int, num: str):
47     """Print num, an n-bit binary, in decimal: numreps b2d type n_bit num.
48     Example: numreps b2d u 6 0b11_1111."""
49     dec = b2d_(b_type, n_bit, num)
50     typer.echo(f'{n_bit}-bit binary {num} to decimal: {dec}')
51
52
53 # n-bit Hexadecimal to decimal
54 @app.command()
55 def h2d(b_type: str, n_bit: int, num: str):
56     """Print num, an n-bit hex, in decimal: numreps h2d type n_bit num.
57     Example: numreps h2d u 6 0b3F."""
58     dec = h2d_(b_type, n_bit, num)
59     typer.echo(f'{n_bit}-bit binary {num} to decimal: {dec}')

```

7.2.2.3 The recipes.py file

This is the file for implementing all commands of the project. When the project grows, we may use a `recipes` folder to host the implementation of commands in multiple files.

`recipes.py`:

```

1  from numreps.representations import (
2      to_dec_twos_complement,
3      add_separator,
4      pad_num_seq,
5      pad_hex_seq,
6      strip_formatters,
7  )
8
9
10 #####
11 # Decimal to binary or hexadecimal
12 def d2b_(n_bit: int, num: int):
13     num = to_dec_twos_complement(n_bit, num)
14     bin_seq = bin(num)[2:]          # Binary sequence without '0b'
15     bin_seq = pad_num_seq(n_bit, bin_seq)
16     seged_bin = '0b' + add_separator(bin_seq, 4)

```

```

17     return seged_bin
18
19
20 def d2h_(n_bit: int, num: int):
21     num = to_dec_twos_complement(n_bit, num)
22     hex_seq = hex(num)[2:].upper() # Hexadecimal seq without '0x'
23     # n_digit = (n_bit - 1) // 4 + 1
24     hex_seq = pad_hex_seq(n_bit, hex_seq)
25     seged_hex = '0x' + add_separator(hex_seq, 4)
26     return seged_hex
27
28
29 #####
30 # Binary or hexadecimal to decimal
31 def b2d_(b_type: str, n_bit: int, num: str):
32     bin_seq = strip_formatters(num, '0b', '_')
33     bin_seq = bin_seq[-n_bit:] # Only keep the last n_bit bits
34     dec_num = int(bin_seq, 2)
35     if (bin_seq[0] == '1') and (b_type == 'i'):
36         dec_num -= 2**n_bit
37     return dec_num
38
39
40 def h2d_(b_type: str, n_bit: int, num: str):
41     hex_seq = strip_formatters(num, '0x', '_')
42     dec_num = int(hex_seq, 16)
43     bin_seq = d2b_(n_bit, dec_num)
44     dec_num = b2d_(b_type, n_bit, bin_seq)
45     return dec_num

```

7.2.2.4 The representations.py file

This is the file hosting the true functionalities that can be used by the CLI App or the GUI App. When the project grows, we may use multiple files or even multiple folders to host the source files.

representations.py.py:

```

1 # Decimal-based two's complement
2 def to_dec_twos_complement(n_bit: int, num: int):
3     mask = 2**n_bit - 1
4     num = num & mask
5     return num
6
7
8 # Add underscore separator in a sequence (seq) every seg_size chars
9 def add_separator(seq: str, seg_size: int):
10     reversed_seq = "".join(reversed(seq))
11     len_seq = len(reversed_seq)
12     num_of_underscores = (len_seq - 1) // seg_size
13
14     index = 0
15     new_seq = ""
16     while (index < num_of_underscores):
17         new_seq += reversed_seq[seg_size * index: seg_size * (index+1)] + '_'
18         index += 1
19     new_seq += reversed_seq[seg_size * index:]
20

```

```

21     return "".join(reversed(new_seq))
22
23
24 # Pad binary sequence
25 def pad_num_seq(n_bit: int, num_seq: str):
26     if len(num_seq) < n_bit:
27         num_seq = '0' * (n_bit - len(num_seq)) + num_seq
28     return num_seq
29
30
31 # Pad hexadecimal sequence
32 def pad_hex_seq(n_bit: int, hex_seq: str):
33     n_digit = (n_bit - 1) // 4 + 1
34     return pad_num_seq(n_digit, hex_seq)
35
36
37 # Binary or hexadecimal to decimal
38 def strip_formatters(num_seq: str, prefix: str, separator: str):
39     num_seq = num_seq.replace(prefix, "")
40     num_seq = num_seq.replace(separator, "")
41     return num_seq

```

7.2.3 Creating source files in the `tests` folder

We use `pytest` to test our functions. Here, we put the test functions into two files:

- `test_recipes.py`: Used to test the API functions hosted in `recipes.py` that can be used by the CLI or GUI apps.
- `test_representations.py`: Used to test the functions hosted in `representations.py`.

7.2.3.1 The `test_recipes.py` file

`test_recipes.py`:

```

1  import pytest
2
3  from numreps.recipes import d2b_, d2h_, b2d_, h2d_
4
5
6  def test_d2b_():
7      assert d2b_(8, 1) == "0b0000_0001"
8      assert d2b_(8, -1) == "0b1111_1111"
9      assert d2b_(9, 17) == "0b0_0001_0001"
10     assert d2b_(9, -17) == "0b1_1110_1111"
11
12
13  def test_d2h_():
14     assert d2h_(8, 1) == "0x01"
15     assert d2h_(8, -1) == "0xFF"
16     assert d2h_(9, 17) == "0x011"
17     assert d2h_(9, -17) == "0x1EF"
18     assert d2h_(18, 17) == "0x0_0011"
19     assert d2h_(18, -17) == "0x3_FFEF"
20
21
22  def test_b2d_():

```

```

23     assert b2d('u', 8, '0b1010_1010') == 170
24     assert b2d('i', 8, '0b1010_1010') == 170 - 256
25     assert b2d('i', 8, '0b0_1010_1010') == 170 - 256
26
27
28 def test_h2d():
29     assert h2d('u', 8, '0xAA') == 170
30     assert h2d('i', 8, '0xAA') == 170 - 256
31     assert h2d('i', 8, '0x0AA') == 170 - 256

```

7.2.3.2 The test_representations.py file

test_representations.py:

```

1  import pytest
2
3  from numreprs.representations import (
4      to_dec_twos_complement,
5      add_separator,
6      pad_num_seq,
7      pad_hex_seq,
8      strip_formatters,
9  )
10
11
12 def test_to_dec_twos_complement():
13     assert to_dec_twos_complement(8, 1) == 1
14     assert to_dec_twos_complement(8, -1) == 255
15     assert to_dec_twos_complement(4, 17) == 1
16     assert to_dec_twos_complement(4, -17) == (256-17) & 15
17
18
19 def test_add_separator():
20     assert add_separator("abcdefghij", 4) == "ab_cdef_ghij"
21     assert add_separator("123", 4) == "123" # No separator needed
22     assert add_separator("12345678", 3) == "12_345_678" # Segments of 3
23     assert add_separator("1", 3) == "1" # Single character, no separator
24     assert add_separator("ABCDEFGH", 2) == "AB_CD_EF_GH" # Segments of 2
25     assert add_separator("A", 2) == "A" # Empty string case
26
27
28 def test_pad_num_seq():
29     # We mainly test the padding of binary sequence here
30     assert pad_num_seq(2, "10") == "10"
31     assert pad_num_seq(8, "1000") == "00001000"
32
33
34 def test_pad_hex_seq():
35     assert pad_hex_seq(4, "a") == "a"
36     assert pad_hex_seq(6, "a") == "0a"
37     assert pad_hex_seq(8, "a") == "0a"
38
39
40 def test_strip_formatters():
41     assert strip_formatters("0b1_1010", "0b", "_") == "11010"
42     assert strip_formatters("0xABC_def9", "0x", "_") == "ABCdef9"
43     assert strip_formatters("0b1_1010", "0b", "_") == "11010"

```

7.3 Adding dependencies

We add two different kind of dependencies for this project.

The first is like Typer, used when both running the App or test various functions. Here, we run `uv add typer`.

The second is like Pytest, used only during developing of the project. Here, we use `uv add --dev pytest` to install it only for the develop group.

7.4 Running the tests

To run the tests of the project, we need to activate the venv of this project and run `pytest` under the root folder of the project.

7.5 Running the project

To run the project as an App, we need to install it first by running `uv tool install . -e`.

After this, we can run the project in anywhere on our computer in a terminal by running `numreps --help` and use the commands list there.

8 Packaging an exemplary project with local dependency

This is an exemplary project for illustrating packaging with uv when there are local dependencies. Please follow the steps below to create the project.

8.1 Creating the project

Go to the parent folder where you want to place this project. Say, `c:\projects` on Windows or `~/projects` on Unix-like systems.

Create the project by running:

```
1 uv init --package --python 3.12 numreps-repl
```

This project contains an app that can run the REPL loop based on the numreps package we develop locally. It uses prompt toolkit for text-based user interface.

8.2 Editing source files

We edit the following source files for this project:

The `README.md` file:

```
1 This project is based on the numreps package we develop locally. It provides a
   text-based user interface so that users can use this App repeatedly without
   exiting.
2
3 To run the code, type `numreps-repl` in a terminal. Follow the hints in the top
   of the screen.
4
5 See the code for details of the functionalities.
```

The `__init__.py` file:

```
1 from numreps_repl.app_tui import app
2
3 def main():
4     app()
```

The `app_tui.py` file:

```
1 #!/usr/bin/env python
2 """A number representation converter."""
3
4 from prompt_toolkit.application import Application
5 from prompt_toolkit.document import Document
6 from prompt_toolkit.key_binding import KeyBindings
7 from prompt_toolkit.layout.containers import HSplit, Window
8 from prompt_toolkit.layout.layout import Layout
9 from prompt_toolkit.styles import Style
10 from prompt_toolkit.widgets import SearchToolbar, TextArea
11
12 from numreps.recipes import d2b_, d2h_, b2d_, h2d_
13
14
15 cvt_states = {"D2B": 0, "D2H": 1, "B2D": 2, "H2D": 3}
16 cvt_state = cvt_states["D2B"] # Initial state
17
18 app_hints = "Commands of the App:\n" + \
19     "- To exit, press Control-Q.\n" + \
20     "- To perform Binary to Decimal conversion, press Control-B.\n" + \
21     "- To perform Decimal to Binary conversion, press Control-C.\n" + \
22     "- To perform Decimal to Hexadecimal conversion, press Control-D.\n" + \
23     "- To perform Hexadecimal to Decimal conversion, press Control-H.\n"
24 app_hint_field = TextArea(style="class:app-field", text=app_hints, height=6)
25
26 cvt_hints = [
27     "Decimal to Binary.      Input: n_bit num (ex: 8 -2)",
28     "Decimal to Hexadecimal. Input: n_bit num (ex: 8 2)",
29     "Binary to Decimal.      Input: type (u/i) n_bit num (ex: i 6 0b11_1010)",
30     "Hexadecimal to Decimal. Input: type (u/i) n_bit num (ex: i 8 0xEF)"
31 ]
32 cvt_cmd_field = TextArea(style="class:cvt-field",
33     text=cvt_hints[cvt_state], height=1)
34
35 output_field = TextArea(style="class:output-field", text="")
36
37 input_field = TextArea(
38     height=1,
39     prompt=">>> ",
40     style="class:input-field",
41     multiline=False,
42     wrap_lines=False,
43     search_field=SearchToolbar(), # For reverse search
44 )
45
46
47 # NOTE: It's better to assign an `accept_handler`, rather than adding a
48 # custom ENTER key binding. This will automatically reset the input
```

```

49 #         field and add the strings to the history.
50 def accept_input(buff):
51     # Evaluate "calculator" expression.
52     try:
53         inputs = input_field.text.split()
54         if cvt_state == cvt_states["D2B"]:
55             n_bit, num = int(inputs[0]), int(inputs[1])
56             output = f"\n\n{n_bit}-bit D2B conversion of {num} ==>\n" + \
57                 f"Out: {d2b_(n_bit, num)}"
58         if cvt_state == cvt_states["D2H"]:
59             n_bit, num = int(inputs[0]), int(inputs[1])
60             output = f"\n\n{n_bit}-bit D2H conversion of {num} ==>\n" + \
61                 f"Out: {d2h_(n_bit, num)}"
62         if cvt_state == cvt_states["B2D"]:
63             b_type, n_bit, num = inputs[0], int(inputs[1]), inputs[2]
64             output = f"\n\n{n_bit}-bit B2D conversion of {num} ==>\n" + \
65                 f"Out: {b2d_(b_type, n_bit, num)}"
66         if cvt_state == cvt_states["H2D"]:
67             b_type, n_bit, num = inputs[0], int(inputs[1]), inputs[2]
68             output = f"\n\n{n_bit}-bit H2D conversion of {num} ==>\n" + \
69                 f"Out: {h2d_(b_type, n_bit, num)}"
70     except BaseException as e:
71         output = f"\n\n{e}"
72
73     output_text = output_field.text + output
74     output_field.buffer.document = Document( # add text to output buffer.
75         text=output_text, cursor_position=len(output_text)
76     )
77
78 input_field.accept_handler = accept_input
79
80 container = HSplit(
81     [
82         app_hint_field,
83         Window(height=1, char="-", style="class:line"),
84         cvt_cmd_field,
85         Window(height=1, char="-", style="class:line"),
86         output_field,
87         Window(height=1, char="-", style="class:line"),
88         input_field,
89     ]
90 )
91
92 style = Style(
93     [
94         ("app-field", "bg:#404000 #ffffff"),
95         ("cvt-field", "bg:#004400 #ffffff"),
96         ("output-field", "bg:#000044 #ffffff"),
97         ("input-field", "bg:#000000 #ffffff"),
98         ("line", "#004400"),
99     ]
100 )
101
102 kb = KeyBindings()
103
104 @kb.add("c-b")
105 def _(event):
106     global cvt_state

```

```
107     cvt_state = cvt_states["B2D"]
108     cvt_cmd_field.buffer.document = Document( text=cvt_hints[cvt_state])
109
110 @kb.add("c-c")
111 def _(event):
112     global cvt_state
113     cvt_state = cvt_states["D2B"]
114     cvt_cmd_field.buffer.document = Document( text=cvt_hints[cvt_state])
115
116 @kb.add("c-d")
117 def _(event):
118     global cvt_state
119     cvt_state = cvt_states["D2H"]
120     cvt_cmd_field.buffer.document = Document( text=cvt_hints[cvt_state])
121
122 @kb.add("c-h")
123 def _(event):
124     global cvt_state
125     cvt_state = cvt_states["H2D"]
126     cvt_cmd_field.buffer.document = Document( text=cvt_hints[cvt_state])
127
128 @kb.add("c-q")
129 def _(event):
130     "Pressing Ctrl-Q or Ctrl-C will exit the user interface."
131     event.app.exit()
132
133 application = Application(
134     layout=Layout(container, focused_element=input_field),
135     key_bindings=kb,
136     style=style,
137     mouse_support=True,
138     full_screen=True,
139 )
140
141 def app():
142     application.run()
```

8.3 Adding dependencies

We also add two different kind of dependencies for this project.

The first is the numreps project we develop locally. To add this dependency, we run `uv add ../numreps`. Note that we have provided the path of this dependency.

The second is the conventional external dependency; here, we run `uv add prompt-toolkit` to add a new package so that we can develop the REPL functionalities in CLI applications.

8.4 Installing the project

We also run `uv tool install . -e` to install the project so that it can run anywhere like a command.

8.5 Running the project

You can run the app in any terminal by executing `numreps-repl`. Follow the on-screen hints at the top of the screen for instructions.